Recursive Language Models for RTL Code Generation

Eduard Anton
McGill University
Montreal, Canada

1 Introduction

The automatic generation of high-quality Register Transfer Level (RTL) code using Large Language Models (LLMs) has recently emerged as a promising research direction. RTL generation is particularly challenging because hardware description languages such as VHDL and Verilog differ significantly from traditional sequential programming languages. They require strong structural reasoning, explicit concurrency, timing semantics, and strict syntactic correctness to successfully compile and simulate.

While modern LLMs demonstrate impressive capabilities in software code generation, several limitations still remain when applying them to RTL design. First, the availability of high-quality training data for hardware description generation is limited, particularly for VHDL. Second, LLMs often lack an explicit structural understanding of hardware architectures, since RTL descriptions often encode non-sequential hardware behavior rather than sequential program execution from traditional programming languages [7]. Third, LLMs tend to hallucinate invalid or inconsistent hardware structures, leading to compilation or simulation failures [5].

Existing approaches to zero-shot RTL generation primarily fall into two categories: pipeline-based reasoning methods [7][5][6] and retrieval-augmented generation (RAG) methods [4][1]. Pipeline approaches attempt to guide the LLM through intermediate reasoning stages that improve the planning of the generated code, while retrieval-based approaches provide expert reference implementations as contextual examples to guide the generation. Although both strategies improve generation quality, they significantly increase context length and verbosity, which raises computational cost and degrades performance due to context noise [6]. Traditionally, LLM systems for long-horizon tasks rely on context summarization or pruning to control context size [9]. However, these methods are lossy and may remove critical information, while still remaining ineffective when intermediate representations themselves are highly verbose.

This project proposes a hybrid architecture combining structured reasoning pipelines with recursive language model [10] execution to improve zero-shot RTL generation for both VHDL and Verilog. The key idea is to separate hardware logic reasoning from deterministic code synthesis, allowing the system to generate structurally correct hardware descriptions while maintaining efficient context usage.

,
2026.

2 Relevant Prior Work

In the zero-shot setting, recent work has explored leveraging powerful pre-trained LLMs to generate RTL code without task-specific fine-tuning. Pipeline-based reasoning methods focus on improving structural planning before code generation. Chain-of-Descriptions (CoD) introduces a multi-stage pipeline where the model first generates a structured plan describing the hardware design, then refines the plan before translating it into RTL code [7]. Similarly, the Paradigm-Based approach classifies problems into hardware design paradigms such as combinational, sequential, or behavioral logic, and applies specialized generation pipelines for each category [5]. While these approaches improve structural correctness and reduce hallucinations, they often rely on rigid predefined pipelines and may struggle with more complex designs.

Retrieval-augmented generation (RAG) approaches aim to improve generation quality through context injection, where expert reference implementations are provided during inference. VeriRAG introduces a hardware-aware retrieval mechanism by training an autoencoder to identify examples with similar hardware structures rather than relying solely on textual similarity [4]. DeepV instead uses a pre-trained semantic embedding model to retrieve contextually similar designs from a hardware corpus. However, RAG-based approaches depend heavily on the quality and diversity of the available corpus, which is particularly challenging due to the limited availability of large public datasets [1]. Furthermore, enforcing language-specific structural constraints may require large amounts of contextual examples. VeriRAG partially addresses this issue by incorporating compiler feedback, allowing the model to iteratively correct syntactic errors during generation [4].

Another, more generic method explores agentic feedback-driven generation. AutoChip integrates compiler and simulator feedback into an iterative generation loop, allowing the system to refine its intermediate implementation drafts based on programmatic feedback results. It also employs a combination of large and small LLMs to balance cost and performance [6]. However, like many of the previous methods, such systems often rely on long context windows and sliding-context strategies to maintain generation coherence, which increases computational overhead and introduces noise in the context.

3 Methodology

The objective of this work is to develop a framework capable of zero-shot generation of RTL code in both VHDL and Verilog using LLMs. The core of the approach relies on Recursive Language Models (RLMs), an inference-time scaffold built around a base LLM that interacts with a persistent external environment through a Read-Eval-Print Loop (REPL). Instead of placing the entire prompt inside the model's context window, the prompt is stored as a variable in the external environment, and the model generates Python code that programmatically accesses and transforms it through iterative execution steps [10]. Within this framework, we separate structural reasoning from code synthesis using two distinct roles. A parent orchestrator model analyzes the hardware specification and produces a structured representation of the design by capturing elements such as module decomposition, control logic, and data flow, similar to pipeline-based approaches such as CoD [7] or paradigm-based generation [5]. The parent model generates Python code that invokes a separate recursive coding model inside the RLM REPL environment. This recursive coding model synthesizes the final RTL implementation by generating HDL code while creating and reusing intermediate variables and invoking recursive sub-calls to the language model as needed. Candidate models for this component include specialized RTL code generation models such as CodeV [11] and RTLCoder [3]. In this way, the system decomposes complex hardware generation tasks into smaller sub-problems and combines their outputs programmatically. Because the parent model only accesses compact metadata about the REPL state rather than the full prompt, the framework mitigates context-window limitations and reduces context rot [10].

In addition to a recursive sub-LLM, the RLM scaffold also enables the integration of external tools within the generation loop. In particular, the system will incorporate compiler and simulator feedback to validate generated RTL implementations. The following tools will be used for compilation and simulation:

- **GHDL:** An open-source VHDL compiler and simulator used to compile generated VHDL code and verify its correctness through simulation.
- **Icarus Verilog:** An open-source Verilog compiler and simulation framework used to validate generated Verilog implementations.

Compiler feedback allows the system to detect syntax errors and structural inconsistencies and iteratively refine the generated code. Although retrieval-augmented generation (RAG) mechanisms could also be integrated into the agentic environment, efficient hardware-specific retrieval is not the primary focus of this project due to the difficulty of constructing high-quality RTL corpora [1].

The system will be evaluated using established benchmarks for zero-shot RTL generation:

- **VHDL-Eval:** A benchmark for evaluating VHDL code generation consisting of 202 hardware design problems derived from translated Verilog tasks and additional VHDL tutorial problems. Each task includes a hardware specification, a canonical solution, and self-verifying testbenches that automatically validate functional correctness of generated VHDL code [8].
- **Verilog-Eval:** A benchmark dataset that evaluates the generation of Verilog modules using functional correctness through compilation and testbench-based verification [2].

Performance will be measured using standard code-generation metrics, including Pass@1, which measures the probability that the first generated sample compiles and passes the provided testbench, and Pass@5, which measures the probability that at least one of five generated samples succeeds. To assess the contribution of the proposed framework, performance will be compared against zero-shot baselines without the RLM scaffold, as well as pipeline-style reasoning baselines such as Chain-of-Descriptions (CoD) [7]. In addition, ablation studies will be conducted to evaluate the impact of compiler feedback within the generation loop. Experiments will be performed using frontier LLMs including Claude 4 and Gemini 3, allowing the analysis of how different models perform within the proposed recursive RTL generation framework.

4 Timeline

Table 1. Project Timeline (9 Weeks)

Week	Tasks / Milestones
Feb 28 – Mar 6	Literature review of prior work.
Mar 7 – Mar 13	Development of the RLM environment and REPL.
Mar 14 – Mar 20	Setup and testing of benchmarks and compiler toolchains.
Mar 21 – Mar 27	Integration of compilers and benchmarks into the RLM framework
Mar 28 – Apr 3	Experiments and data collection for Verilog generation and preparation of intermediate results for the progress presentation.
Apr 4 – Apr 10	Experiments and data collection for VHDL generation and initial draft of the project report.
Apr 11 – Apr 17	Final data collection, baseline comparisons, and ablation studies.
Apr 18 – Apr 24	Final analysis of results and completion of the main report sections.
Apr 25 – May 1	Final report revision

References

[1] Zahin Ibnat, Paul E. Calzada, Rasin Mohammed Ihtemam, Sujan Kumar Saha, Jingbo Zhou, Farimah Farahmandi, and Mark Tehranipoor. 2025. DeepV: A Model-Agnostic Retrieval-Augmented Framework for Verilog Code Generation with a High-Quality Knowledge Base. arXiv:2510.05327 [cs.AR] https://arxiv.org/abs/2510.05327

[2] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. arXiv:2309.07544 [cs.LG] https://arxiv.org/abs/2309.07544

[3] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution. arXiv:2312.08617 [cs.PL] doi:10.1109/LAD62341.2024.10691788

[4] Haomin Qi, Yuyang Du, Lihao Zhang, Soung Chang Liew, Kexin Chen, and Yining Du. 2026. VeriRAG: A Retrieval-Augmented Framework for Automated RTL Testability Repair. arXiv:2507.15664 [cs.AR] https://arxiv.org/abs/2507.15664

[5] Wenhao Sun, Bing Li, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, and Ulf Schlichtmann. 2025. Paradigm-Based Automatic HDL Code Generation Using LLMs. arXiv:2501.12702 [cs.PL] https://arxiv.org/abs/2501.12702

[6] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887 [cs.PL] https://arxiv.org/abs/2311.04887

[7] Prashanth Vijayaraghavan, Apoorva Nitsure, Charles Mackin, Luyao Shi, Stefano Ambrogio, Arvind Haran, Viresh Paruthi, Ali Elzein, Dan Coops, David Beymer, Tyler Baldwin, and Ehsan Degan. 2024. Chain-of-Descriptions: Improving Code LLMs for VHDL Code Generation and Summarization. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '24)*. ACM, 1–10. doi:10.1145/3670474.3685966

[8] Prashanth Vijayaraghavan, Luyao Shi, Stefano Ambrogio, Charles Mackin, Apoorva Nitsure, David Beymer, and Ehsan Degan. 2024. VHDL-Eval: A Framework for Evaluating Large Language Models in VHDL Code Generation. arXiv:2406.04379 [cs.SE] https://arxiv.org/abs/2406.04379

[9] Howard Yen, Ashwin Paranjape, Mengzhou Xia, Thejas Venkatesh, Jack Hessel, Danqi Chen, and Yuhao Zhang. 2025. Lost in the Maze: Overcoming Context Limitations in Long-Horizon Agentic Search. arXiv:2510.18939 [cs.CL] https://arxiv.org/abs/2510.18939

[10] Alex L. Zhang, Tim Kraska, and Omar Khattab. 2026. Recursive Language Models. arXiv:2512.24601 [cs.AI] https://arxiv.org/abs/2512.24601

[11] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. 2025. CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization. arXiv:2407.10424 [cs.PL] https://arxiv.org/abs/2407.10424